

Sinchai - Smart Irrigation Decision Support System

Sinchai is a smart irrigation controller and decision support system for semi-arid smallholdings, built with Node.js and React (Ink). It combines soil moisture monitoring, Open-Meteo weather forecasts, and crop water requirements to make explainable irrigation decisions, conserve water through rain-skips, and expose farm state via the Model Context Protocol (MCP).

Prerequisites

- Node.js >= 22.13 (built-in `node:sqlite` DatabaseSync)
- Modern terminal supporting ANSI colors (recommended >= 100x30)

Installation

```
npm ci
```

Running the System

```
# Launch interactive terminal UI (Ink / React)
npm start

# Launch scripted demo mode (accelerated clock, simulated weather & dropouts)
npm run demo

# Run controller loop headlessly
npm run headless

# Run environment and connectivity diagnostics
npm run doctor

# Start Model Context Protocol (MCP) server over stdio
npm run mcp

# Run full Vitest test suite
npm test
```

Keyboard Controls

- `[d]` Dashboard: Top status bar, 3-row zone table with ASCII progress bars, weather panel, and alerts feed.
- `[z]` Zones: Selected zone details, ASCII sparkline of last 24h readings, and decision tree trace.
- `[r]` Reports: Water consumption vs baseline (30 min/day) and stress hours.
- `[l]` Rain-check: Ledger summary and list of rain skips with HIT/MISS verdicts.

- `[1-3]` Select Zone 1, 2, or 3.
- `[v]` Toggle valve for selected zone (manual override).
- `[m]` Toggle system mode between AUTO and MANUAL.
- `[a]` Acknowledge active alerts.
- `[q]` Quit application.

Decision Engine & Formulas

The engine evaluates zones in strict priority order:

1. System mode check (MANUAL yields NO_OP).
2. Sensor fault check (raw ADC readings outside 50–4045 mV range trigger FAULT).
3. Sensor freshness check (readings older than 15 minutes trigger SENSOR_OFFLINE).
4. Waterlogging check (moisture $\geq 100\%$ for ≥ 3 hours triggers WATERLOGGED).
5. Rain-skip check (forecast rain ≥ 5.0 mm at probability $\geq 70\%$ within 12 hours yields SKIP_RAIN).
6. High wind check (wind ≥ 25 km/h yields WAIT_WIND for sprinkler and flood).
7. Cooldown check (minimum 120 minutes between irrigation events).
8. Restricted irrigation window (enforces permitted diurnal windows).
9. Target deficit calculation and runtime duration.

Duration Formula

$$\text{deficit} = \frac{\text{target_pct} - \text{moisture}}{100} \times \text{water_holding_mm}$$

$$\text{minutes} = \frac{\text{deficit}}{\text{flow_mm_hr} \times \text{efficiency}} \times 60$$

Subject to the maximum runtime safety cap (180 minutes).

Model Context Protocol (MCP)

Sinchai includes an MCP server running over stdio, implementing the official `@modelcontextprotocol/sdk`.

Available Tools

- `list_zones` : List all zones with crop, flow rate, irrigation type, moisture, reading age, and valve state.
- `get_zone_status(zone_id)` : Get current status, recommendation, and active alerts for one zone.
- `explain_recommendation(zone_id)` : Show calculation steps and thresholds behind a recommendation.
- `get_weather_outlook` : Get current weather, 12-hour forecast, and cache age.
- `water_report(days)` : Water use report per zone with litres used, baseline, and stress hours.
- `rain_check_ledger(limit)` : List recent rain skips with verdicts and summary statistics.

- `request_valve(zone_id, action, minutes)` : Request manual valve operation (gated by `allow_valve_requests`).

Client Configuration

Add this configuration to Claude Desktop or another MCP client:

```
{
  "mcpServers": {
    "sinchai": {
      "command": "node",
      "args": ["--no-warnings", "/Users/dhruvkhare/Desktop/smart-irrigation-system/src/mcp-se",
      "env": {
        "SINCHAI_DB": "/Users/dhruvkhare/Desktop/smart-irrigation-system/data/sinchai.db"
      }
    }
  }
}
```

Project Structure

```
smart-irrigation-system/
├── package.json          # Node.js package configuration and dependencies
├── src/
│   ├── alerts.js        # Alert generation, cooldowns, and lifecycle
│   ├── cli.js           # Command-line interface with node:util parseArgs
│   ├── clock.js         # Real time and accelerated demo time
│   ├── config.js        # Configuration loader and SQLite settings applicator
│   ├── db.js            # node:sqlite DatabaseSync operations and WAL mode
│   ├── defaults.json    # Baseline farm configuration
│   ├── demo.js          # Scripted demo scenario runner
│   ├── demo-scenario.json # Demo scenario weather and sensor events
│   ├── engine.js        # Pure decision engine
│   ├── farm.js          # Controller loop and tick handler
│   ├── index.js         # CLI entry point
│   ├── ledger.js        # Rain-check skip ledger and HIT/MISS resolution
│   ├── mcp-server.js    # Model Context Protocol stdio server
│   ├── reports.js       # Water consumption and stress reports
│   ├── schema.sql       # SQLite schema
│   ├── sensors.js       # Sensor calibration math and Box-Muller simulation
│   ├── valves.js        # Valve safety limits and runtime tracking
│   ├── weather.js       # Open-Meteo client with caching
│   └── ui/
│       ├── app.jsx      # Ink application container and keyboard input
│       ├── ascii.js     # ASCII progress bars and sparklines
│       ├── dashboard.jsx # Status bar, zone table, weather panel, alerts feed
│       ├── rain-check.jsx # Rain-check ledger view
│       ├── reports.jsx   # Water use reports view
│       └── zone_view.jsx # Zone detail and sparkline view
└── tests/
    ├── calibration.test.js # Sensor calibration math tests
    ├── db.test.js         # Database integrity and WAL tests
    ├── engine.test.js     # Decision engine tests
    ├── farm.test.js       # Headless controller acceptance tests
    ├── ledger.test.js     # Rain-check ledger HIT/MISS tests
    ├── mcp.test.js        # MCP stdio server tests
    ├── reports.test.js    # Water volume and stress calculations
    ├── sensors.test.js    # Simulated sensor tests
    ├── weather.test.js    # Weather parsing and cache tests
    └── ui.test.jsx        # Ink terminal UI frame tests
```

AI Assistance

This project was built with the assistance of an AI coding assistant (Google DeepMind Antigravity).

The complete, chronological record of all prompts, development iterations, and assistant responses is documented in [AI CHAT TRANSCRIPT.md](#).